Southern New Hampshire University

Andres Trujillo
CS 410 Software Reverse Engineering
6/21/2026

CS 410 Project Two Security Report

Function	Block of C++ Code	Identified Security Vulnerability
CheckUserPermissionAccess( )	if (password == "123") return true;	You can't hardcode passwords in plaintext. Anyone looking at the compiled binary or source code can easily extract "123". In a real-world application, we need to hash and salt passwords, store them in a secure database, and compare the hashes during login instead of using raw strings.
CheckUserPermissionAccess( )	cin >> username; // ... if (password == "123")	Prompt for the username but never actually verify it against anything? Right now, someone can type anything for the username and still get access. Proper authentication requires validating both the identity (username) and the secret (password) together.
CheckUserPermissionAccess( )	string password; // ... cin >> password;	We are storing the inputted password in a standard std::string. After the function finishes, the string is destroyed, but the actual characters might linger in the computer's RAM until that memory is overwritten. In highly secure environments, malware or memory dumps can scrape these leftover passwords. We'd normally use specialized secure allocators that scrub the memory with zeroes immediately after use.
ChangeClientChoice()	cin >> clientNumber;	Function 'cin >>' when

	// ... if (clientNumber < 1 || clientNumber > 5) return;	reading directly into an integer variable. If the user types a letter (like 'A'), cin enters a fail state and leaves the invalid character in the buffer. We need to check if (cin.fail()), clear the error state, and flush the buffer, otherwise it can cause the program to behave unpredictably.

main()	while (!CheckUserPermissionAccess())
	
	{
	
	}
		Login loop is insecure because there is no limit on how many times CheckUserPermissionAccess( ) can be called, an attacker can script an automated dictionary attack to guess the password infinitely.

main()	cin >> selection; // ... } while (selection != 3);	Similar to the input validation issue above, if a user inputs a string instead of a number for the menu selection, cin fails and this do-while loop will spin infinitely. This essentially causes a crash or a localized Denial of Service (DoS) by locking up the CPU. Sanitize and validate user input is a must.
Global Scope	string clients[5] = { ... }; int services[5] = { ... };	We have client PII (Personally Identifiable Information) hardcoded globally in plaintext. It's ok for a quick mock-up or school project, but in production, we never want sensitive customer data living statically in memory to avoid memory scraping attacks. This data should accessed dynamically from a secure database.

Security Vulnerabilities

1. Hardcoded Password

The following code contains a hardcoded password:

```
if (password == "123")
  return true;
```

This is a security vulnerability because the application's authentication credential is embedded directly into the source code. Anyone with access to the source code can immediately discover the password. Even if the source code is not available, an attacker may be able to extract the password by reverse engineering the compiled executable using tools that inspect strings stored within the binary. Because the password never changes unless the software is modified and recompiled, it becomes a single point of failure for the entire authentication system.

In production environments, passwords should never be stored or compared as plaintext values. Instead, user passwords should be hashed using a secure cryptographic algorithm and stored in a protected database. During login, the user's entered password is hashed and compared against the stored hash. This approach significantly reduces the risk of credential exposure and prevents attackers from easily recovering passwords if the application or database is compromised.

2. Username Is Not Validated

The code prompts the user for a username but does not verify it:

```
cin >> username;

// ...

if (password == "123")
```

This is a security vulnerability because authentication requires verification of both a user's identity and their secret credential. Although the program collects a username, it never checks whether that username exists or whether it is associated with the supplied password. As a result, any user can enter an arbitrary username and gain access as long as they know the password.

This weakness defeats the purpose of user authentication because access control is based solely on knowledge of a single password. In a real application, usernames and passwords should be validated together against a trusted data source such as a database or directory service. Proper authentication ensures that users can only access resources associated with their own accounts and cannot impersonate other users by simply entering a different username.

3. Password Stored in Standard Memory

The code stores the password in a standard string object:

```
string password;
cin >> password;
```

This code presents a security concern because sensitive information is stored in ordinary memory without any special protections. When a password is read into a std::string, the password remains in memory for as long as the string exists. Even after the variable goes out of scope, the memory may still contain the password until it is overwritten by another process or allocation.

Although this vulnerability is less severe than a hardcoded password, it can become significant in high-security environments. Memory dumps, debugging tools, malware, or forensic analysis may reveal passwords that remain in memory after use. Security-sensitive applications often use specialized memory handling techniques that overwrite sensitive data immediately after authentication to reduce the risk of credential exposure.

4. Improper Input Validation for Client Selection

The following code accepts user input directly into an integer variable:

```
cin >> clientNumber;

if (clientNumber < 1 || clientNumber > 5)
  return;
```

This code attempts to validate the range of values entered by the user but fails to verify whether the input operation itself succeeded. If a user enters a non-numeric value such as a letter or symbol, the input stream enters a fail state. When this occurs, the variable may retain its previous value while the invalid data remains in the input buffer.

Improper input validation is one of the most common software vulnerabilities because unexpected input can cause unpredictable program behavior. Attackers frequently exploit weak input validation to trigger logic errors, bypass intended controls, or crash applications. Proper validation should check whether the input operation succeeded before processing the value and should clear the input stream when invalid data is detected.

5. Unlimited Authentication Attempts

The application repeatedly prompts for login credentials using the following loop:

```
while (!CheckUserPermissionAccess())
{
}
```

This creates a security vulnerability because there is no limit on the number of failed login attempts. An attacker can repeatedly submit passwords until the correct one is discovered.

This type of attack is known as a brute-force attack and is one of the most common methods used to compromise weak authentication systems.

Modern authentication systems typically implement protections such as account lockouts, increasing delays between failed attempts, CAPTCHA verification, or monitoring systems that detect suspicious login behavior. These safeguards make it significantly more difficult for attackers to automate password guessing. Without such controls, even a moderately strong password may eventually be discovered through repeated attempts.

6. Infinite Loop Caused by Invalid Menu Input

The menu selection logic accepts input directly from the user:

```
cin >> selection;

} while (selection != 3);
```

This code assumes that the user will always enter a valid numeric value. However, if a user enters text instead of a number, the input stream enters a fail state and subsequent input operations may stop functioning correctly. Since the program never checks for input failure, the loop may continue executing indefinitely without giving the user an opportunity to correct the mistake.

This issue can lead to a localized Denial of Service (DoS) condition. A denial-of-service vulnerability occurs when legitimate users are prevented from using an application as intended. While this vulnerability may not affect an entire network or system, it can make the application unusable and force users to terminate and restart the program. Proper input validation and error handling are necessary to prevent this type of failure.

7. Hardcoded Client Data in Global Variables

The application stores client information directly in global arrays:

```
string clients[5] = { ... };
int services[5] = { ... };
```

This code presents a security concern because potentially sensitive information is embedded directly into the application's memory space. Any data compiled into the executable can potentially be recovered through reverse engineering, memory inspection tools, debugging utilities, or accidental disclosure of source code. If the information contains customer records, account details, or other personally identifiable information (PII), unauthorized disclosure could result in privacy violations.

In professional software systems, sensitive data is typically stored in secured databases rather than hardcoded into the application. Databases provide access controls, auditing capabilities, encryption options, and centralized management of information. Separating

application code from sensitive data reduces the risk of exposing confidential information and makes the system easier to maintain and update securely.

Recommendations

1. Replace the Hardcoded Password with Secure Authentication

The hardcoded password should be removed entirely from the source code. Instead of comparing user input to a plaintext string stored within the application, user credentials should be stored in a secure database. Passwords should never be stored in plaintext; they should be hashed using a strong cryptographic hashing algorithm and compared against the stored hash during authentication. This ensures that even if the database is compromised, attackers cannot easily recover the original passwords.

For a simple educational application, the password could be stored externally in a configuration file rather than compiled into the executable. In a production environment, a complete authentication system should be implemented that uses salted password hashes, secure storage mechanisms, and proper access controls. Removing hardcoded credentials eliminates one of the most common and dangerous authentication weaknesses.

2. Validate Usernames Along with Passwords

The application should verify both the username and password rather than checking only the password. Each username should correspond to a valid account, and the supplied password should be verified against that account's stored credentials. Authentication should only succeed when both pieces of information match a legitimate user record.

Implementing proper username validation prevents users from impersonating others by entering arbitrary usernames. A common approach is to store usernames and password hashes in a database or other secure data source. During login, the application retrieves the record associated with the entered username and verifies the provided password against the stored credential. This ensures that access is granted only to authorized users.

3. Protect Sensitive Password Data in Memory

Sensitive password information should be stored in memory for the shortest amount of time possible. After authentication is completed, the memory containing the password should be cleared so that the credential cannot remain accessible to memory inspection tools, debugging utilities, or malware. While this may not be critical for a small classroom project, it is an important security practice in applications that handle sensitive information.

One approach is to overwrite the memory containing the password immediately after it is no longer needed. Some security-focused libraries provide specialized containers designed to securely erase sensitive information when the object is destroyed. By minimizing the amount of

time passwords remain in memory, the risk of credential exposure through memory analysis is significantly reduced.

4. Implement Proper Input Validation for Numeric Values

All user input should be validated before it is used by the application. When reading numeric values with cin, the program should verify that the input operation succeeded before processing the value. If invalid data is entered, the application should clear the error state, remove the invalid input from the buffer, and prompt the user to enter a valid value.

For example, the program can use cin.fail() to detect invalid input and then recover by calling cin.clear() and cin.ignore(). Proper input validation prevents unexpected program behavior and ensures that users cannot accidentally or intentionally place the application into an invalid state. This defensive programming technique improves both security and reliability.

5. Limit Failed Login Attempts

The login system should enforce a maximum number of failed authentication attempts. After a predefined number of unsuccessful login attempts, the application should either terminate the session, temporarily lock the account, or require an additional verification step before allowing further attempts. These controls make automated password guessing attacks significantly more difficult.

Many modern authentication systems also introduce delays between failed login attempts. Even a short delay can dramatically reduce the effectiveness of brute-force attacks by limiting the number of passwords an attacker can test within a given period. Implementing account lockout policies and rate limiting provides an important layer of defense against unauthorized access attempts.

6. Add Error Handling for Menu Selections

The menu system should verify that user input is valid before using it to control program flow. If the user enters text instead of a number, the application should detect the error, display an informative message, clear the input stream, and request new input rather than continuing execution with invalid data.

Adding proper error handling prevents the application from entering an infinite loop or becoming unresponsive. The program should continue prompting the user until a valid menu option is entered. This approach improves the user experience while also preventing accidental denial-of-service conditions caused by malformed input.

7. Move Client Data to Secure External Storage

Client information should not be hardcoded directly into the application's source code or stored in global variables. Instead, the data should be retrieved from a secure external data source such as a database, encrypted file, or other managed storage solution. This separates sensitive information from the application itself and makes it easier to manage, update, and protect.

Using external storage also allows security controls such as access restrictions, encryption, auditing, and backup procedures to be applied to the data. If customer information must be stored, only authorized users and processes should be able to access it. Keeping sensitive data outside the executable reduces the risk of exposing confidential information through source code disclosure, reverse engineering, or memory inspection.